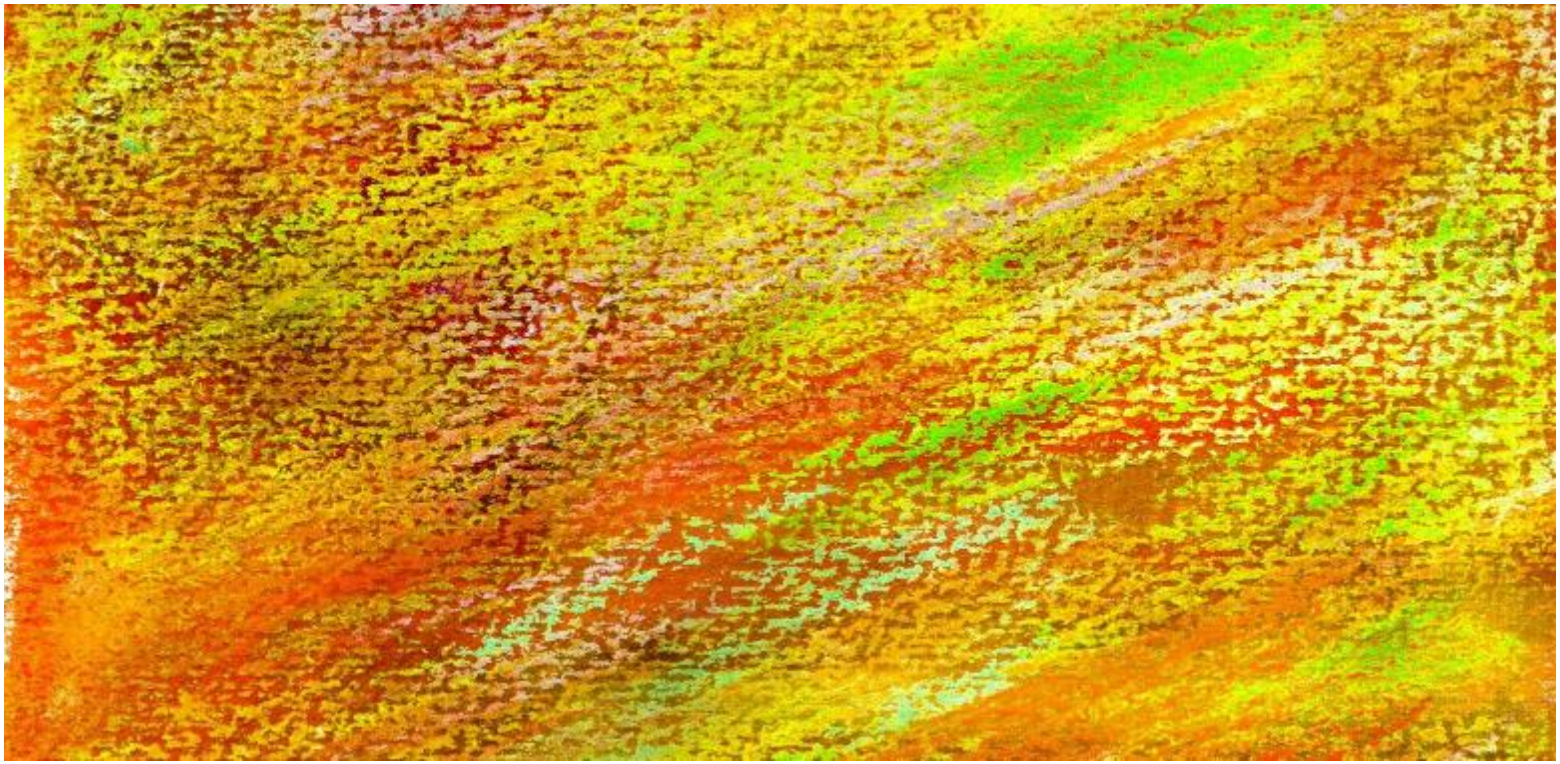
DAVID M. KROENKE and DAVID J. AUER

DATABASE CONCEPTS, 7th Edition

Chapter Six

Structured Query Language



Chapter Objectives

- Learn basic SQL statements for creating database structures
- Learn basic SQL statements for adding data to a database
- Learn basic SQL SELECT statements and options for processing a single table
- Learn basic SQL SELECT statements for processing multiple tables with subqueries
- Learn basic SQL SELECT statements for processing multiple tables with joins
- Learn basic SQL statements for modifying and deleting data from a database
- Learn basic SQL statements for modifying and deleting database tables and constraints

Structured Query Language

- **Structured Query Language**
  - Acronym: SQL
  - Pronounced as “S-Q-L” [“Ess-Que-El”]
  - Originally developed by IBM as the SEQUEL language in the 1970s
  - SQL-92 is an ANSI national standard adopted in 1992.
  - SQL:2011 is current standard.

SQL Defined

- SQL is not a programming language, but rather a data sublanguage.
- SQL is comprised of
  - Data definition language (DDL)
    - Used to define database structures
  - Data manipulation language (DML)
    - Data definition and updating
    - Data retrieval (Queries)
  - SQL/Persistent Stored Modules (SQL/PSM)
    - Procedural programming capabilities [See Appendix E]
  - Transaction control language (TCL)
    - Control transaction behavior [See Chapter 6]
  - Data control language (DLC)
    - Grant and revoke database permissions [See Chapter 6]

SQL for Data Definition

- The SQL data definition statements include:
  - CREATE
    - To create database objects
  - ALTER
    - To modify the structure and/or characteristics of database objects
  - DROP
    - To delete database objects
  - TRUNCATE
    - To delete table data while keeping structure

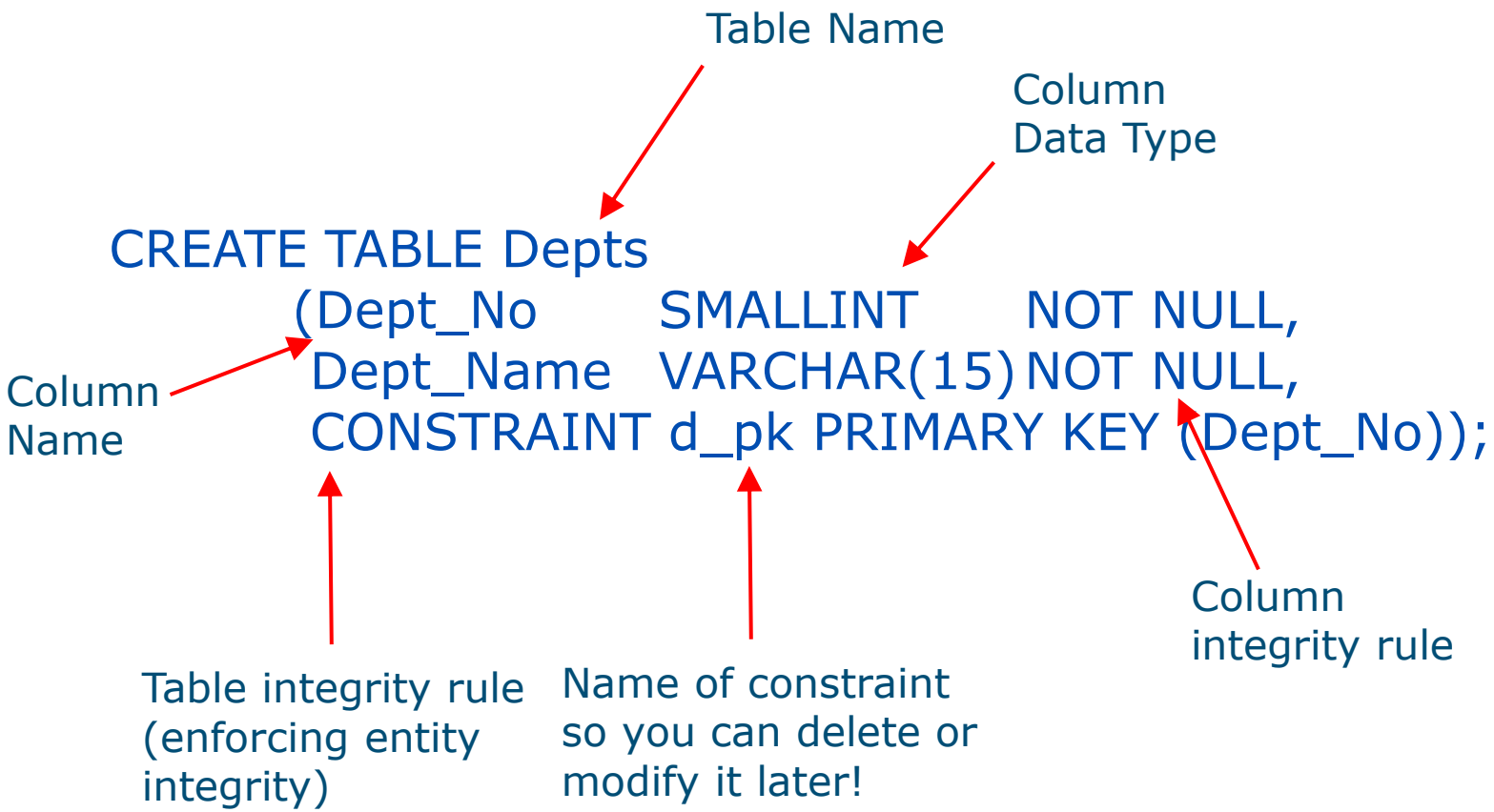
Main Data Types

Keyword	Description	Sample Declaration	Sample Values
CHAR	Fixed length character string.	CHAR (4)	‘ABCD’,’ ‘,AbCd’
VARCHAR	Variable length character string (you only specify max length).	VARCHAR (4)	‘ABCD’,’Ab’,’Add’
BIT	Fixed length bit string.	BIT (4)	‘1010’,’1111’,’0101’
BIT VARYING	Variable length bit string	BIT VARYING (4)	’01’,’1111’,’010’
NUMERIC	Decimal number.	NUMERIC (5) NUMERIC (5,2)	12345 12345.01
DECIMAL	Higher precision decimal num.	DEC (4)	12345
INTEGER	Whole number (4 bytes).	INTEGER	12345678
SMALLINT	Whole number (2 bytes).	SMALLINT	1234
FLOAT	Floating point of at least the stated precision.	FLOAT (4)	0.1234E+05
REAL	Floating point with implementation-defined precision.	REAL	0.1234E+05
DOUBLE PRECISION	Floating point with higher precision than REAL.	DOUBLE PRECISION	0.1234E+05
DATE	Calendar date consisting of DAY, MONTH & YEAR	DATE	’02-MAY-05’

Modified from Howe (2001) Fig. 19.3, page 230.

Example: CREATE TABLE with Primary Key

Specify at table level on creation:



Example: CREATE TABLE with Primary Key

Specify at attribute level on creation:

Column integrity rule

```
CREATE TABLE Depts
(Dept_No SMALLINT     CONSTRAINT d_pk PRIMARY KEY,
 Dept_Name     VARCHAR(15)   NOT NULL);
```

Specify at table level post-hoc using ALTER TABLE:

```
CREATE TABLE Depts
        (Dept_No        SMALLINT,
         Dept_Name      VARCHAR(15)   NOT NULL);

ALTER TABLE Depts
        ADD CONSTRAINT d_pk PRIMARY KEY (Dept_No);
```

Post hoc table integrity rule

SQL for Data Definition: CREATE

- Creating database tables
  - The SQL CREATE TABLE statement

```
CREATE TABLE EMPLOYEE(
   EmpID        Integer        PRIMARY KEY,
   EmpName      Char(25)       NOT NULL
   );
```

SQL for Data Definition: CREATE with CONSTRAINT I

- Creating database tables with PRIMARY KEY constraints
  - The SQL CREATE TABLE statement
  - The SQL CONSTRAINT keyword

```
CREATE TABLE EMPLOYEE(
   EmpID        Integer     NOT NULL,
   EmpName      Char(25)    NOT NULL,
   CONSTRAINT   Emp_PK      PRIMARY KEY(EmpID)
   );
```

SQL for Data Definition: CREATE with CONSTRAINT II

- Creating database tables with composite primary keys using PRIMARY KEY constraints
  - The SQL CREATE TABLE statement
  - The SQL CONSTRAINT keyword

```
CREATE TABLE EMP_SKILL(
    EmpID        Integer       NOT NULL,
    SkillID      Integer       NOT NULL,
    SkillLevel   Integer       NULL,
    CONSTRAINT   EmpSkill_PK   PRIMARY KEY
                                  (EmpID, SkillID)
     );
```

SQL for Data Definition: CREATE with CONSTRAINT III

- Creating database tables using PRIMARY KEY and FOREIGN KEY constraints
  - The SQL CREATE TABLE statement
  - The SQL CONSTRAINT keyword

```
CREATE TABLE EMP_SKILL(
    EmpID        Integer       NOT NULL,
    SkillID      Integer       NOT NULL,
    SkillLevel   Integer       NULL,
    CONSTRAINT   EmpSkill_PK   PRIMARY KEY
                                  (EmpID, SkillID),
   CONSTRAINT    Emp_FK        FOREIGN KEY(EmpID)
                    REFERENCES    EMPLOYEE(EmpID),
    CONSTRAINT   Skill_FK      FOREIGN KEY(SkillID)
                    REFERENCES    SKILL(SkillID)
    );
```

SQL for Data Definition: CREATE with CONSTRAINT IV

- Creating database tables using PRIMARY KEY and FOREIGN KEY constraints
  - The SQL CREATE TABLE statement
  - The SQL CONSTRAINT keyword
  - ON UPDATE CASCADE and ON DELETE CASCADE

```
CREATE TABLE EMP_SKILL(
   EmpID        Integer        NOT NULL,
   SkillID      Integer        NOT NULL,
   SkillLevel   Integer        NULL,
   CONSTRAINT   EmpSkill_PK    PRIMARY KEY(EmpID, SkillID),
   CONSTRAINT   Emp_FK         FOREIGN KEY(EmpID)
                   REFERENCES   EMPLOYEE(EmpID)
                       ON DELETE CASCADE,
   CONSTRAINT   Skill_FK       FOREIGN KEY(SkillID)
                   REFERENCES   SKILL(SkillID)
                       ON UPDATE CASCADE
   );
```

Process SQL CREATE TABLE Statements: Microsoft SQL Server 2014

The SQL script in the tabbed script window

The objects representing the tables created by the script are shown in the expanded Tables folder—*dbo* stands for *database owner*

Messages are shown here—either that the commands were successful or appropriate error messages

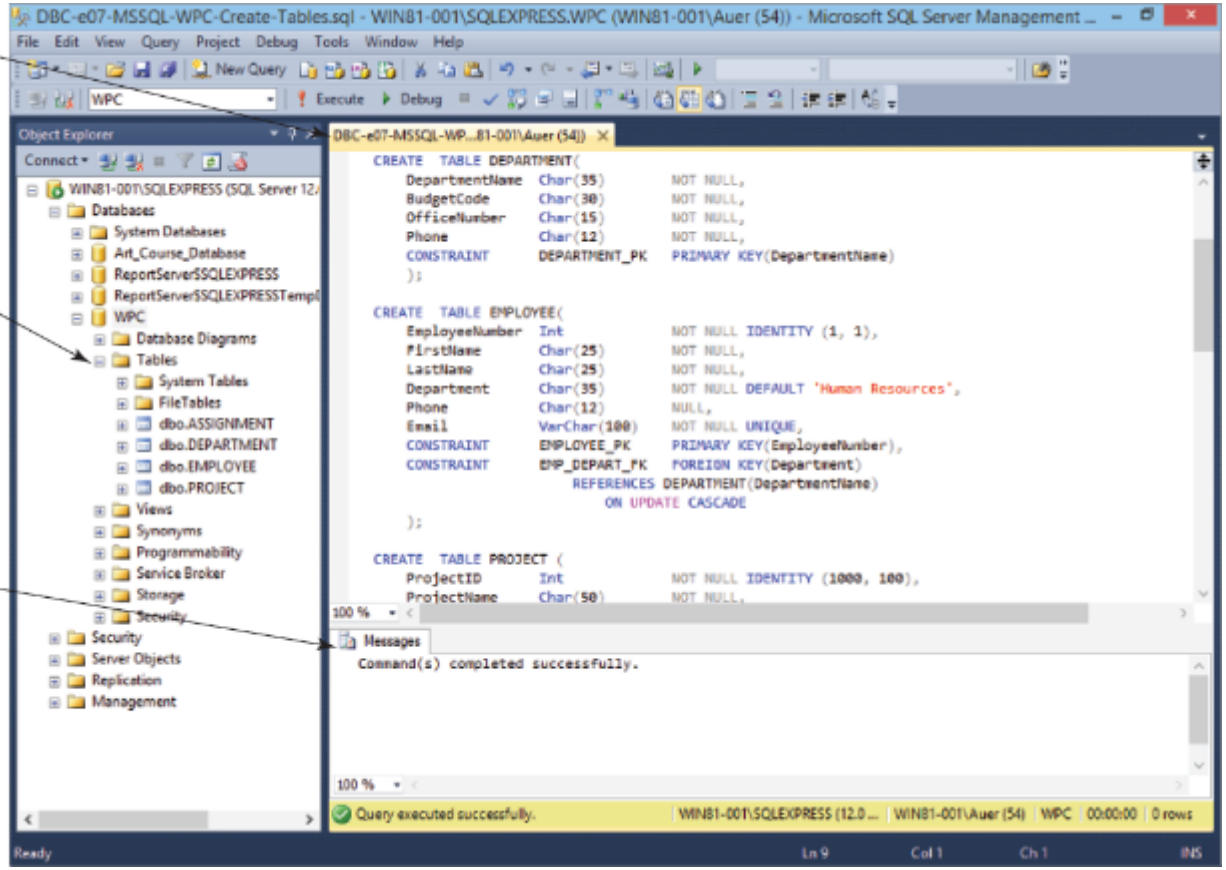


Table 3-8:
Processing the CREATE TABLE Statements Using SQL Server 2014

Process SQL CREATE TABLE Statements: Oracle Database 11*g* Release 2

The SQL script in the tabbed **SQL Worksheet** window

The objects representing the tables created by the script are shown in the expanded Tables folder

Messages are shown here—either that the commands were successful or appropriate error messages

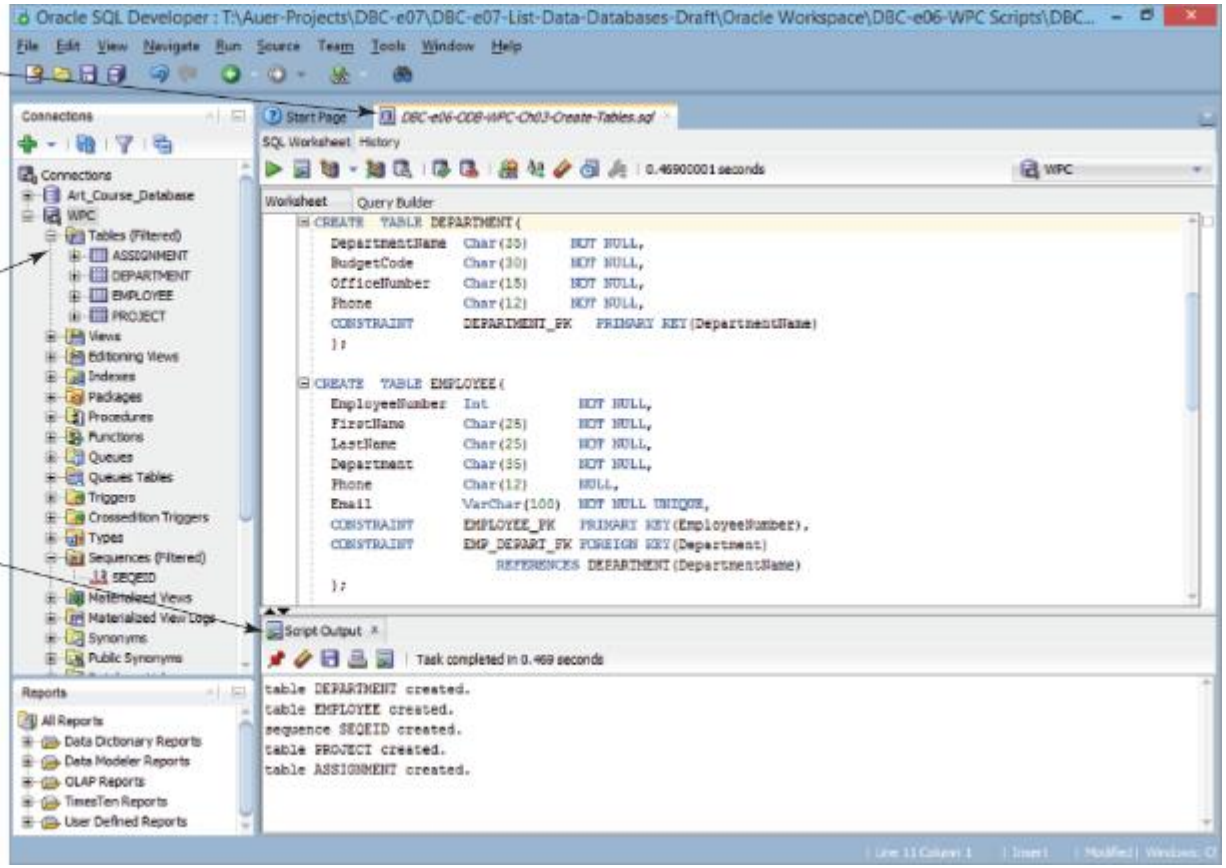


Figure 3-9: Processing the CREATE TABLE Statements Using Oracle Database Express Edition 11*g* Release 2

Process SQL CREATE TABLE Statements: MySQL 5.6

The SQL script in the tabbed Script window

The objects representing the tables created by the script are shown in the expanded *wpc* schema

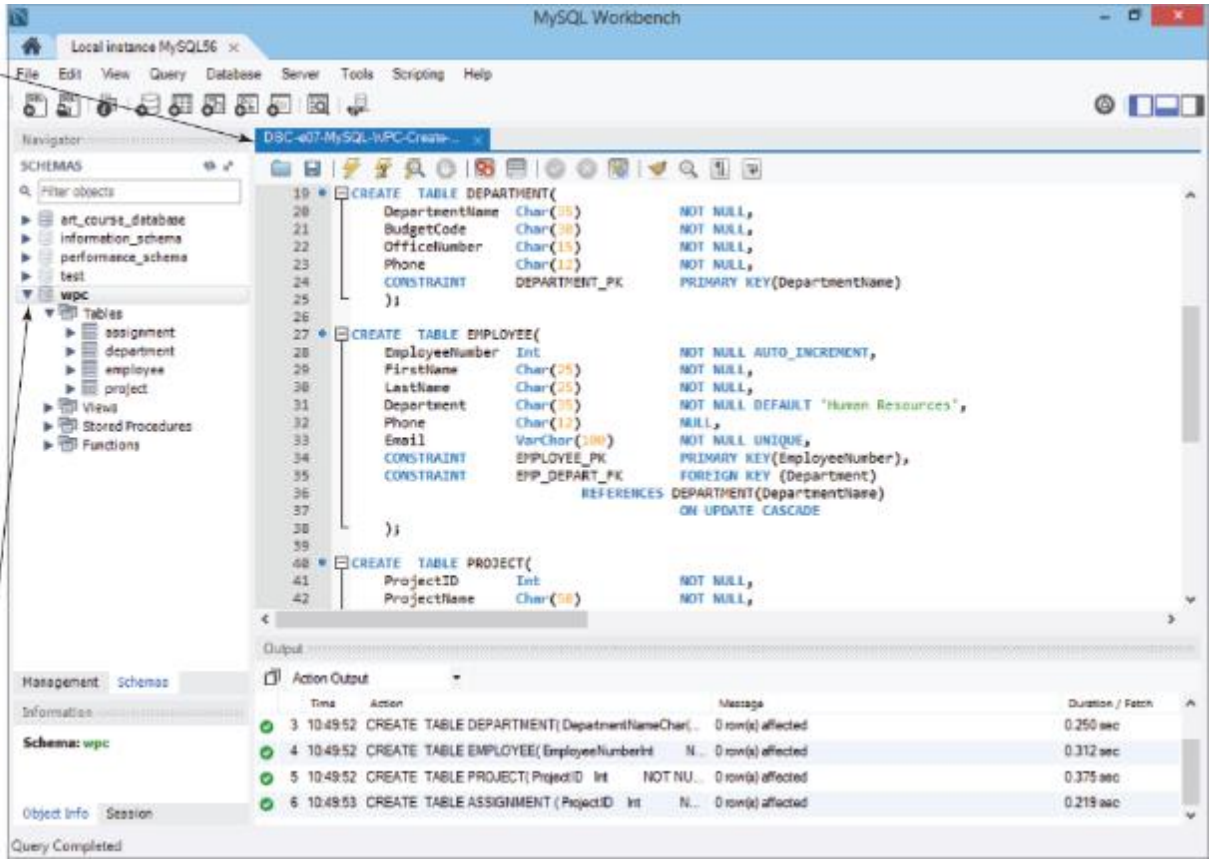


Figure 3-10:
Processing the CREATE TABLE Statements Using MySQL 5.6

Database Diagram in the Microsoft SQL Server Management Studio

The database tables and the links between them are shown in the tabbed Diagram window

The object representing the database diagram is shown in the expanded Database Diagrams folder—*dbo* stands for *database owner*

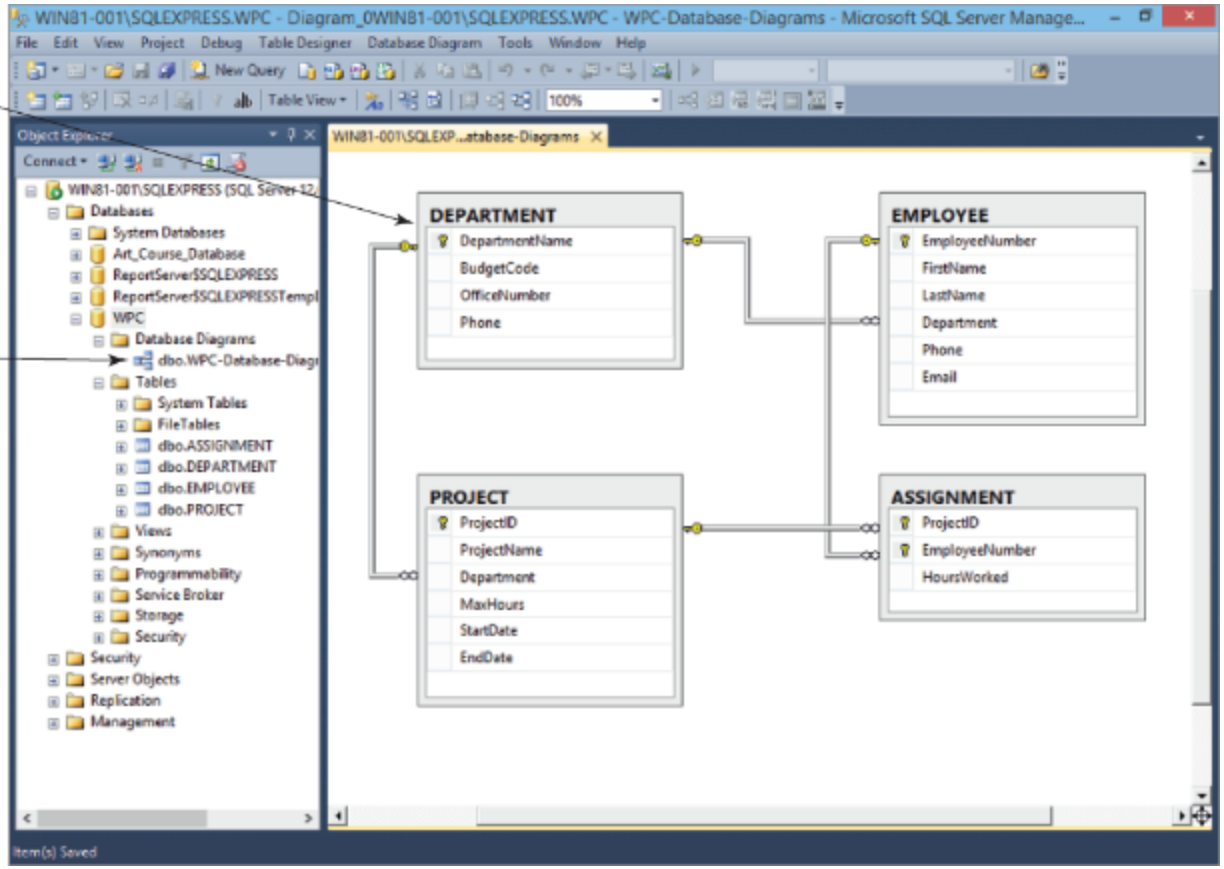


Figure 3-11:
Database Diagram in the Microsoft SQL Server Management Studio

Primary Key Constraint: ALTER I

- Adding primary key constraints to an existing table
  - The SQL ALTER statement

```
ALTER TABLE EMPLOYEE
   ADD CONSTRAINT Emp_PK PRIMARY KEY(EmpID);
```

Composite Primary Key Constraints: ALTER II

- Adding a composite primary key constraint to an existing table
  - The SQL ALTER statement

```
ALTER TABLE EMP_SKILL
   ADD CONSTRAINT EmpSkill_PK
          PRIMARY KEY(EmpID, SkillID);
```

Foreign Key Constraint: ALTER III

- Adding foreign key constraints to an existing table
  - The SQL ALTER statement

```
ALTER TABLE EMPLOYEE ADD
   CONSTRAINT  Emp_FK      FOREIGN KEY(DeptID)
               REFERENCES  DEPARTMENT(DeptID);
```

Adding Data: INSERT

- To add a row to an existing table, use the INSERT statement.
- Non-numeric data must be enclosed in straight (') single quotes.

```
INSERT INTO EMPLOYEE VALUES(91, 'Smither', 12);

INSERT INTO EMPLOYEE (EmpID, SalaryCode)
       VALUES (62, 11);
```

END FOR
CREATING DATABASE &
TABLE STRUCTURE &
INSERT DATA

SQL for Data Retrieval: Queries

- SELECT is the best known SQL statement.
- SELECT will retrieve information from the database that matches the specified criteria using the SELECT/FROM/WHERE framework.

```
SELECT EmpName
FROM   EMPLOYEE
WHERE  EmpID = 2010001;
```

SQL for Data Retrieval: The Results of a Query Is a Relation

- A query pulls information from one or more relations and creates (temporarily) a new relation.
- This allows a query to:
  - Create a new relation
  - Feed information to another query (as a "sub-query")

SQL for Data Retrieval: Displaying All Columns

- To show all of the column values for the rows that match the specified criteria, use an asterisk (*).

```
SELECT    *
FROM      EMPLOYEE;
```

SQL for Data Retrieval: Showing Each Row Only Once

- The DISTINCT keyword may be added to the SELECT statement to inhibit duplicate rows from displaying.

```
SELECT   DISTINCT DeptID
FROM     EMPLOYEE;
```

SQL for Data Retrieval: Specifying Search Criteria

- The WHERE clause stipulates the matching criteria for the record that is to be displayed.

```
SELECT      EmpName
FROM        EMPLOYEE
WHERE       DeptID = 15;
```

Processing SQL Query Statements: Microsoft SQL Server 2014

The **New Query** button

The **Execute** button

The SQL statement in the tabbed query window

The query results

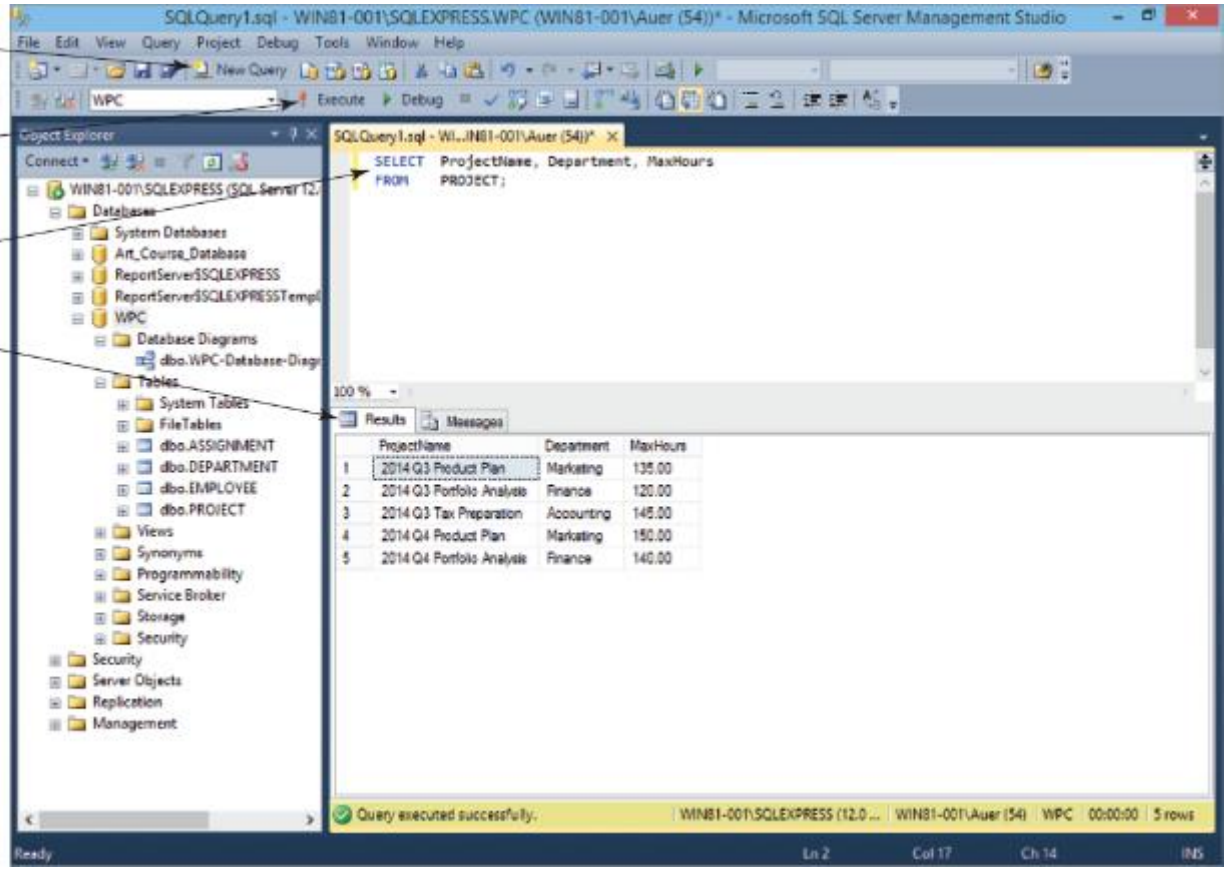


Figure 3-13:
SQL Query Results in the Microsoft SQL Server Management Studio

Processing SQL Query Statements:
Oracle Database Express Edition 11*g* Release 2

The **WPC** tabbed SQL Worksheet window

The **Run Statement** button

The SQL statement in the tabbed SQL Worksheet window

The query results in the tabbed **Query Result** window

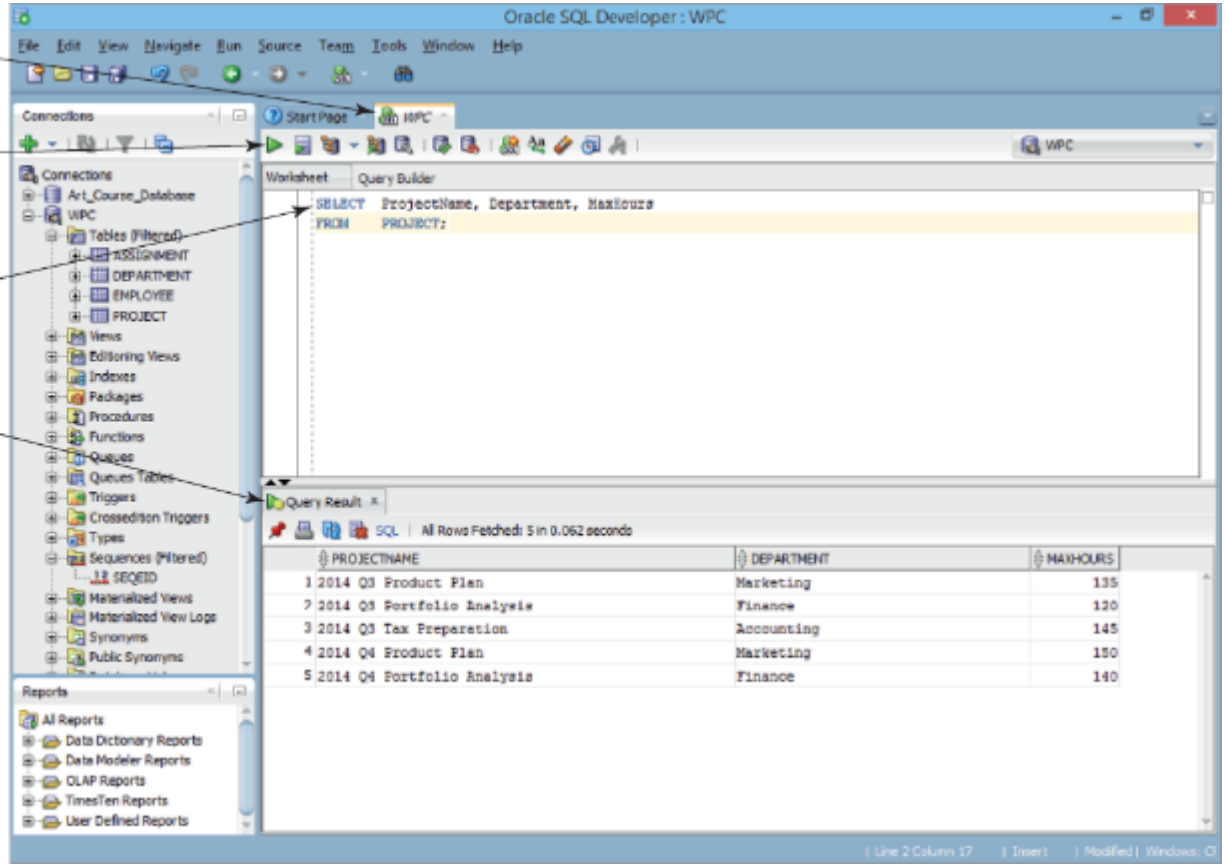


Figure 3-14: SQL Query Results in the Oracle SQL Developer

Processing SQL Query Statements: MySQL 5.6

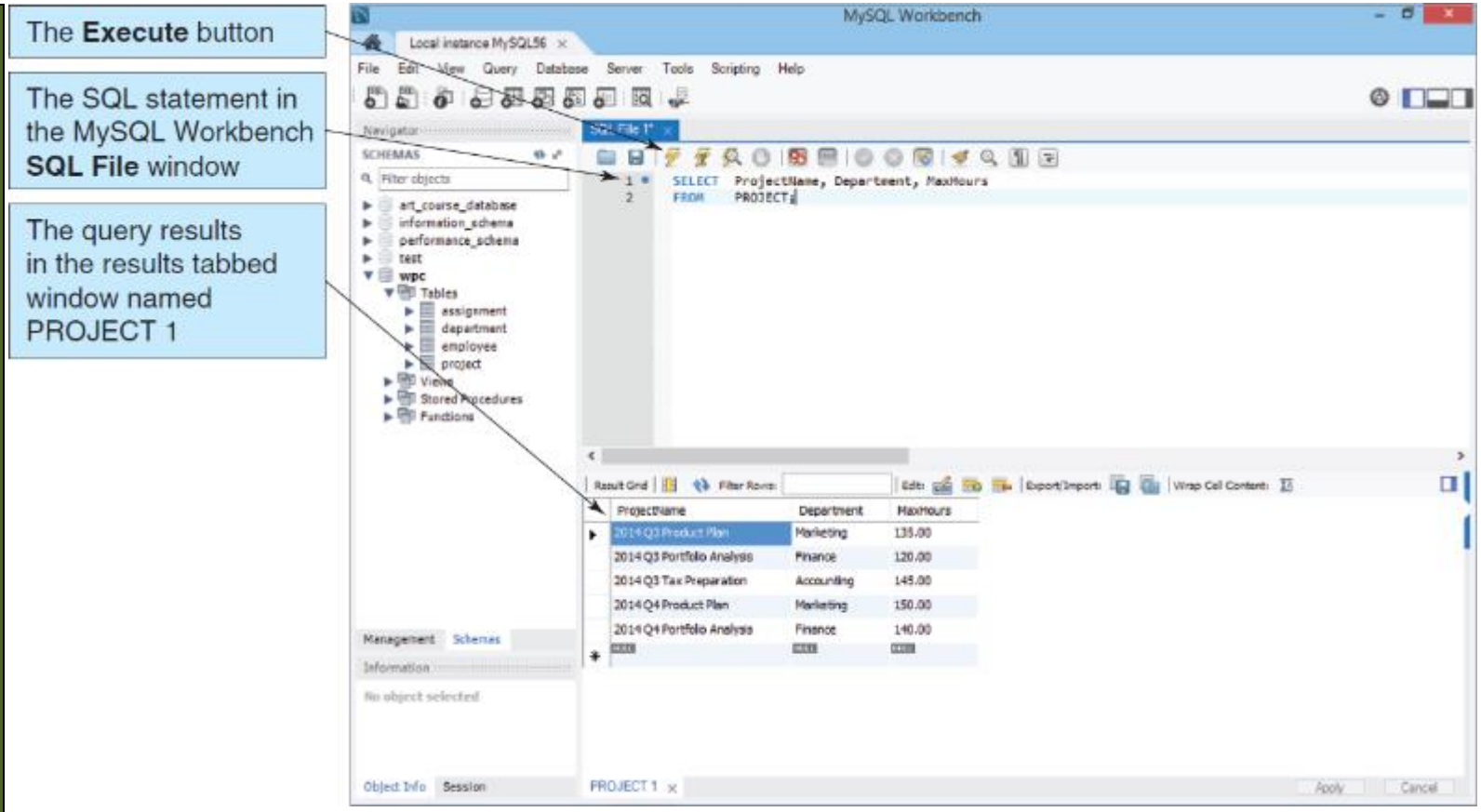


Figure 3-15: SQL Query Results in the MySQL Workbench

SQL for Data Retrieval: Match Criteria

- The WHERE clause match criteria may include
  - Equals “=”
  - Not Equals “<>”
  - Greater than “>”
  - Less than “<”
  - Greater than or Equal to “>=”
  - Less than or Equal to “<=”

SQL for Data Retrieval: Match Operators

- Multiple matching criteria may be specified using
  - AND
    - Representing an intersection of the data sets
  - OR
    - Representing a union of the data sets

SQL for Data Retrieval: Operator Examples

```
SELECT      EmpName
FROM        EMPLOYEE
WHERE       DeptID < 7
            OR     DeptID > 12;
```

```
SELECT      EmpName
FROM        EMPLOYEE
WHERE       DeptID = 9
            AND  SalaryCode <= 23;
```

SQL for Data Retrieval: A List of Values

- The WHERE clause may include the IN keyword to specify that a particular column value must be included in a list of values.

```
SELECT    EmpName
FROM      EMPLOYEE
WHERE     DeptID IN (4, 8, 9);
```

SQL for Data Retrieval: The Logical NOT Operator

- Any criteria statement may be preceded by a NOT operator, which is to say that all information will be shown except that information matching the specified criteria

```
SELECT    EmpName
FROM      EMPLOYEE
WHERE     DeptID NOT IN (4, 8, 9);
```

SQL for Data Retrieval: Finding Data in a Range of Values

- SQL provides a BETWEEN keyword that allows a user to specify a minimum and maximum value on one line.

```
SELECT   EmpName
FROM     EMPLOYEE
WHERE    SalaryCode BETWEEN 10 AND 45;
```

SQL for Data Retrieval: Allowing for Wildcard Searches

- The SQL LIKE keyword allows searches on partial data values.
- LIKE can be paired with wildcards to find rows matching a string value.
  - Multiple character wildcard character is a percent sign (%).
  - Single character wildcard character is an underscore (_).

SQL for Data Retrieval: Wildcard Search Examples

```
SELECT      EmpID
FROM        EMPLOYEE
WHERE       EmpName LIKE 'Kr%';

SELECT      EmpID
FROM        EMPLOYEE
WHERE       Phone LIKE '616-___-____';
```

SQL for Data Retrieval: Sorting the Results

- Query results may be sorted using the ORDER BY clause.

```
SELECT      *
FROM        EMPLOYEE
ORDER BY    EmpName;
```

SQL for Data Retrieval: Built-in SQL Functions

- SQL provides several built-in functions:
  - COUNT
    - Counts the number of rows that match the specified criteria
  - MIN
    - Finds the minimum value for a specific column for those rows matching the criteria
  - MAX
    - Finds the maximum value for a specific column for those rows matching the criteria

SQL for Data Retrieval: Built-in SQL Functions (Cont'd)

- SUM
  - Calculates the sum for a specific column for those rows matching the criteria
- AVG
  - Calculates the numerical average of a specific column for those rows matching the criteria

SQL for Data Retrieval: Built-in Function Examples

```
SELECT COUNT(DeptID)
FROM   EMPLOYEE;

SELECT MIN(Hours) AS MinimumHours,
       MAX(Hours) AS MaximumHours,
       AVG(Hours) AS AverageHours
FROM   PROJECT
WHERE  ProjID > 7;
```

SQL for Data Retrieval: Providing Subtotals: GROUP BY

- Subtotals may be calculated by using the GROUP BY clause.
- The HAVING clause may be used to restrict which data is displayed.

```
SELECT      DeptID,
            COUNT(*) AS NumOfEmployees
FROM        EMPLOYEE
GROUP BY    DeptID
HAVING      COUNT(*) > 3;
```

SQL for Data Retrieval:
Retrieving Information from Multiple Tables

- Subqueries
  - As stated earlier, the result of a query is a relation. As a result, a query may feed another query. This is called a *subquery.*
- Joins
  - Another way of combining data is by using a *join* .
    - Join [also called an Inner Join]
    - Left Outer Join
    - Right Outer Join

SQL for Data Retrieval: Subquery Example

```
SELECT EmpName
FROM   EMPLOYEE
WHERE  DeptID in
       (SELECT  DeptID
       FROM     DEPARTMENT
       WHERE    DeptName LIKE 'Account%');
```

SQL for Data Retrieval: Join Example

```
SELECT   EmpName
FROM     EMPLOYEE AS E, DEPARTMENT AS D
WHERE    E.DeptID = D.DeptID
         AND  D.DeptName LIKE 'Account%';
```

SQL for Data Retrieval: JOIN…ON Example

- The JOIN…ON syntax can be used in joins.
- It has the advantage of moving the JOIN syntax into the FROM clause.

```
SELECT   EmpName
FROM     EMPLOYEE AS E JOIN DEPARTMENT AS D
         ON  E.DeptID = D.DeptID
WHERE    D.DeptName LIKE 'Account%';
```

SQL for Data Retrieval: OUTER JOIN I

- The OUTER JOIN syntax can be used to obtain data that exists in one table without matching data in the other table.

STUDENT

StudentPK	StudentName	LockerFK
1	Adams	NULL
2	Buchanan	NULL
3	Carter	10
4	Ford	20
5	Hoover	30
6	Kennedy	40
7	Roosevelt	50
8	Truman	60

LOCKER

LockerPK	LockerType
10	Full
20	Full
30	Half
40	Full
50	Full
60	Half
70	Full
80	Full
90	Half

(a) The STUDENT and LOCKER Tables Aligned to Show Row Relationships

Figure 3-17: Types of Joins

SQL for Data Retrieval: OUTER JOIN II

Only the rows where LockerFK=LockerPK are shown—Note that some StudentPK and some LockerPK values are not in the results

StudentPK	StudentName	LockerFK	LockerPK	LockerType
3	Carter	10	10	Full
4	Ford	20	20	Full
5	Hoover	30	30	Half
6	Kennedy	40	40	Full
7	Roosevelt	50	50	Full
8	Truman	60	60	Half

(b) INNER JOIN of the STUDENT and LOCKER Tables

Figure 3-17: Types of Joins

SQL for Data Retrieval: OUTER JOIN III

All rows from STUDENT are shown, even where there is no matching LockerFK=LockerPK value

StudentPK	StudentName	LockerFK	LockerPK	LockerType
1	Adams	NULL	NULL	NULL
2	Buchanan	NULL	NULL	NULL
3	Carter	10	10	Full
4	Ford	20	20	Full
5	Hoover	30	30	Half
6	Kennedy	40	40	Full
7	Roosevelt	50	50	Full
8	Truman	60	60	Half

(c) LEFT OUTER JOIN of the STUDENT and LOCKER Tables

Figure 3-17: Types of Joins

SQL for Data Retrieval: OUTER JOIN IV

All rows from LOCKER are shown, even where there is no matching LockerFK=LockerPK value

StudentPK	StudentName	LockerFK	LockerPK	LockerType
3	Carter	10	10	Full
4	Ford	20	20	Full
5	Hoover	30	30	Half
6	Kennedy	40	40	Full
7	Roosevelt	50	50	Full
8	Truman	60	60	Half
NULL	NULL	NULL	70	Full
NULL	NULL	NULL	80	Full
NULL	NULL	NULL	90	Half

(d) RIGHT OUTER JOIN of the STUDENT and LOCKER Tables

Figure 3-17: Types of Joins

SQL for Data Retrieval: LEFT OUTER JOIN Example

- The OUTER JOIN syntax can be used to obtain data that exists in one table without matching data in the other table.

```
SELECT   EmpName
FROM     EMPLOYEE AS E
         LEFT JOIN DEPARTMENT AS D
                ON  E.DeptID = D.DeptID
WHERE    D.DeptName LIKE 'Account%';
```

SQL for Data Retrieval: RIGHT OUTER JOIN Example

- The unmatched data displayed can be from either table, depending on whether RIGHT JOIN or LEFT JOIN is used.

```
SELECT   EmpName
FROM     EMPLOYEE AS E
         RIGHT JOIN DEPARTMENT AS D
                ON  E.DeptID = D.DeptID
WHERE    D.DeptName LIKE 'Account%';
```

Modifying Data using SQL

- Insert
  - Will add a new row in a table (already discussed above)
- Update
  - Will update the data in a table that matches the specified criteria
- Delete
  - Will delete the data in a table that matches the specified criteria

Modifying Data using SQL: Changing Data Values: UPDATE

- To change the data values in an existing row (or set of rows) use the Update statement.

```
UPDATE    EMPLOYEE
SET       Phone '791-555-1234'
WHERE     EmpID = 29;

UPDATE    EMPLOYEE
SET       DeptID = 44
WHERE     EmpName LIKE 'Kr%';
```

Modifying Data using SQL: MERGE

- SQL:2003 introduced the MERGE statement.
  - Combines INSERT and UPDATE into one statement
  - Uses the equivalent of IF-THEN-ELSE logic to decide whether to use INSERT or UPDATE
  - An advanced feature—learn to use INSERT and UPDATE separately first, then consult DBMS documentation

Modifying Data using SQL: Deleting Data: DELETE

- To delete a row or set of rows from a table use the DELETE statement.

```
DELETE FROM EMPLOYEE
WHERE  EmpID = 29;

DELETE FROM EMPLOYEE
WHERE  EmpName LIKE 'Kr%';
```

Modifying Data using SQL: Deleting Database Objects: DROP

- To remove unwanted database objects from the database, use the SQL DROP statement.
- Warning… The DROP statement will permanently remove the object and all data.

```
DROP TABLE EMPLOYEE;
```

Modifying Data using SQL:
Removing a Constraint: ALTER & DROP

- To change the constraints on existing tables, you may need to remove the existing constraints before adding new constraints.

```
ALTER TABLE EMPLOYEE
      DROP CONSTRAINT EmpFK;
```

Modifying Data Using SQL: The CHECK Constraint

- The CHECK constraint can be used to create sets of values to restrict the values that can be used in a column.

```
ALTER TABLE PROJECT
      ADD CONSTRAINT PROJECT_Check_Dates
            CHECK (StartDate < EndDate);
```

SQL Views

- A **SQL View** is a virtual table created by a DBMS-stored SELECT statement that can combine access to data in multiple tables and even in other views.
- SQL views are discussed online in Appendix E.

END OF CHAPTER 6